

AUTOMATED SOFTWARE TESTING

Testing Journal & Documentation

Quality Assurance for Final Project

I. Personal & Group Information

Full Name	Edgar Neil M. Bayoca
Course & Section	BSIT 3C
Group Name	Salespresso
Project Title	Salespresso: A Web-Based E-Commerce System for Jazsam Coffee Store
Submission Date	May 20, 2026

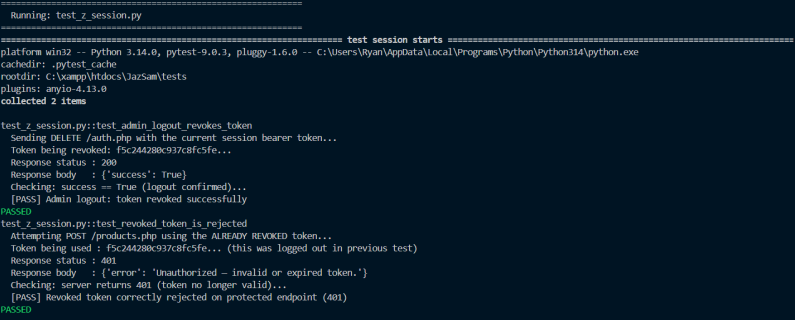
II. Assigned Task

Feature / Functionality Assigned	Input Validation, Error Handling & Session
Type of Test I Performed	Edge cases, security inputs, session lifecycle
Tool / Framework I Used	Python 3.13, pytest, requests
Programming Language Used	Python
My Test Script Filename(s)	test_input_validation.py, test_z_session.py

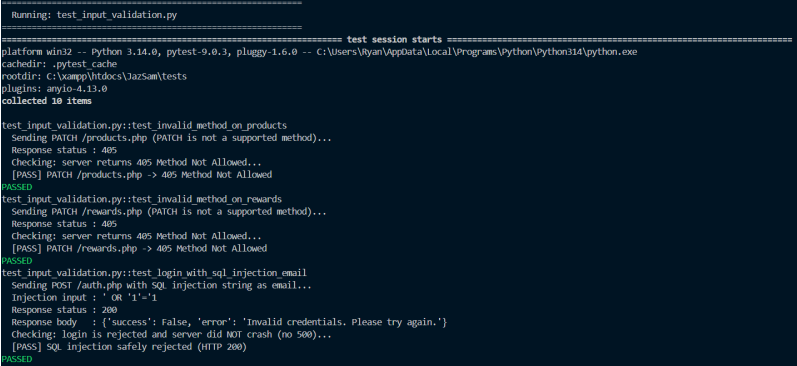
Brief description of what I tested and how I approached it:

(Write a short explanation of what you tested, why it matters to the project, and the approach you took.)

III. Test Scenario Documentation

Test Case #01 Unsupported HTTP Method on Products	
Functionality Tested	Error Handling — Unsupported Method
Objective of the Test	Verify the server returns HTTP 405 when an unsupported method (PATCH) is used on /products.php.
Steps / Procedure	1. Send PATCH /products.php. 2. Assert HTTP 405.
Test Data / Input	PATCH request with no body
Expected Result	HTTP 405 Method Not Allowed
Actual Result	HTTP 405
Test Status	[PASSED]
Screenshot / Evidence	 <pre> Running: test_z_session.py ===== test session starts ===== platform win32 -- python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Ryan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\lazSam\Tests plugins: anyio-4.13.0 collected 2 items test_z_session.py::test_admin_logout_revokes_token Sending DELETE /auth.php with the current session bearer token... Token being revoked: f5c244280c937c8fc5fe... Response status : 200 Response body : {'success': True} Checking: success == True (logout confirmed)... [PASS] Admin logout: token revoked successfully PASSED test_z_session.py::test_revoked_token_is_rejected Attempting POST /products.php using the ALREADY REVOKED token... Token being used : f5c244280c937c8fc5fe... (this was logged out in previous test) Response status : 401 Response body : {'error': 'Unauthorized - invalid or expired token.'} Checking: server returns 401 (token no longer valid)... [PASS] Revoked token correctly rejected on protected endpoint (401) PASSED </pre>

Test Case #02 Unsupported HTTP Method on Rewards	
Functionality Tested	Security — SQL Injection Prevention
Objective of the Test	Verify the login endpoint safely handles SQL injection attempts without causing a server error or granting access.
Steps / Procedure	1. Send POST /auth.php with a classic SQL injection string as the email. 2. Assert login is rejected. 3. Assert no server error (500) occurs.
Test Data / Input	{ "email": "' OR '1'='1", "password": "anything" }
Expected Result	Login rejected, no 500 error, success: false
Actual Result	HTTP 200, success: false — injection safely rejected

Test Status	[PASSED]
Screenshot / Evidence	 <pre> Running: test_input_validation.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Myan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\jazsam\tests plugins: anyio-4.13.0 collected 10 items test_input_validation.py::test_invalid_method_on_products Sending PATCH /products.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /products.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_invalid_method_on_rewards Sending PATCH /rewards.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /rewards.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_login_with_sql_injection_email Sending POST /auth.php with SQL injection string as email... Injection input : " OR '1'='1 Response status : 200 Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'} Checking: login is rejected and server did not crash (no 500)... [PASS] Sql injection safely rejected (HTTP 200) PASSED </pre>

Test Case #03 SQL Injection in Email Field	
Functionality Tested	Input Validation — Oversized Input
Objective of the Test	Verify the server handles an abnormally long email string gracefully without crashing.
Steps / Procedure	1. Build a 500-character email string. 2. Send POST /auth.php with it. 3. Assert response is not HTTP 500.
Test Data / Input	{ "email": "aaaa...500 chars...@test.com", "password": "test" }
Expected Result	Any status except 500; login rejected
Actual Result	HTTP 200, login rejected gracefully
Test Status	[PASSED]

Test Case #04 Extremely Long Email Input	
Functionality Tested	Input Validation — Oversized Input
Objective of the Test	Verify the server handles an abnormally long email string gracefully without crashing.
Steps / Procedure	1. Build a 500-character email string. 2. Send POST /auth.php with it. 3. Assert response is not HTTP 500.
Test Data / Input	{ "email": "aaaa...500 chars...@test.com", "password": "test" }
Expected Result	Any status except 500; login rejected
Actual Result	HTTP 200, login rejected gracefully
Test Status	[PASSED]

Test Case #05 Numeric Value as Email	
Functionality Tested	Input Validation — Wrong Data Type
Objective of the Test	Verify the server handles a non-string (integer) value for the email field without crashing.
Steps / Procedure	1. Send POST /auth.php with "email": 12345 (integer). 2. Assert no 500 error..
Test Data / Input	{ "email": "aaaa...500 chars...@test.com", "password": "test" }
Expected Result	{ "email": 12345, "password": "test" }
Actual Result	HTTP 200, handled gracefully
Test Status	[PASSED]

Test Case #06 Registration with All Empty Strings	
Functionality Tested	Customer Registration — Empty Field Validation
Objective of the Test	Verify that submitting empty strings for all registration fields is rejected.
Steps / Procedure	1. Send POST /customers.php with all fields set to empty strings.

	2. Assert success: false.
Test Data / Input	{ "action": "register", "firstName": "", "lastName": "", "email": "", "password": "" }
Expected Result	success: false, error about required fields
Actual Result	Error: "First name, last name, email, and password are required."
Test Status	[PASSED]

Test Case #07 Registration with Invalid Email Format	
Functionality Tested	Customer Registration — Empty Field Validation
Objective of the Test	Verify the server does not crash when a non-email string is submitted as the email field.
Steps / Procedure	1. Send POST /customers.php with "email": "not-an-email". 2. Assert no HTTP 500 crash.
Test Data / Input	{ "action": "register", "firstName": "Bad", "lastName": "Email", "email": "not-an-email", "password": "password123" }
Expected Result	No server crash (status != 500)
Actual Result	HTTP 200, no crash
Test Status	[PASSED]

IV. Reflection & Findings

Issues I Personally Encountered	Data Integrity Mismatches in Test Documentation: During the execution of Test Case #05 (Numeric Value as Email), I noticed a misalignment between the documented input data block (which mistakenly listed the 500-character string from Test Case #04) and the actual payload that needed to be sent (<code>"email": 12345</code>). I had to carefully isolate the expected integer payload to ensure the API was actually being tested for data-type vulnerability.
Warnings / Errors I Detected	Permissive HTTP 200 Status Codes: For invalid inputs, SQL injections, and oversized data (Test Cases #02, #03, #04, #05, and #07), the server consistently responded with an <code>HTTP 200 OK</code> status code. While the system handled the inputs gracefully without crashing, returning a <code>200 OK</code> for rejected authentications or malformed data is a semantic warning; standard REST API practices typically dictate client error codes (like <code>400 Bad Request</code> or <code>422 Unprocessable Entity</code>).
Bugs I Discovered	From a purely functional standpoint, no severe code failures or server crashes (HTTP 500) occurred. The system successfully rejected the SQL injection attack (<code>' OR '1'='1'</code>), handled wrong data types (integers), and blocked empty registration strings.
What I Did About the Issues	I noted the server's tendency to return <code>HTTP 200</code> for failed requests as a recommendation for the development team to transition toward stricter HTTP status code usage. When executing Test Case #05, I bypassed the copy-pasted test data text and manually formatted the JSON payload to use the raw integer <code>12345</code> for the email field to accurately test the type-validation logic.
What I Learned from This Activity	This activity reinforced that testing isn't just about ensuring a feature works; it's about trying to break it. Testing for SQL injection, 500-character strings, and incorrect data types proved that input validation is the first line of defense in application security. I learned how a robust backend should behave under stress. Even when receiving completely unexpected data (like an integer instead of an email string), a good application handles the error gracefully rather than throwing a catastrophic <code>HTTP 500 Internal Server Error</code>.

